

Lecture-#9

Natural Language Processing

A8706

Syntax Analysis

Shift Reduce Parsing

- ❖ To build a parser, we need an algorithm that can perform the steps in the above rightmost derivation for any grammar and for any input string.
- ❖ Every CFG turns out to have an automata that is equivalent to it, called pushdown automata (just like regular expression can be converted to finite-state automata).
- ❖ An algorithm for parsing that is general for any given CFG and input string.
- ❖ The algorithm is called shift-reduce parsing which uses two data structures: a buffer for input symbols and a stack for storing CFG symbols.

Parsing Algorithms

This parser requires some data structures i.e.

An input buffer for storing the input string.

A stack for storing and accessing the production rules.

Basic Operations –

Shift: This involves moving symbols from the input buffer onto the stack.

Reduce: If the handle appears on top of the stack then, its reduction by using appropriate production rule is done i.e. RHS of a production rule is popped out of a stack and LHS of a production rule is pushed onto the stack.

Accept: If only the start symbol is present in the stack and the input buffer is empty then, the parsing action is called accept. When accepted action is obtained, it means successful parsing is done.

Error: This is the situation in which the parser can neither perform shift action nor reduce action and not even accept action.

Parsing Algorithms

Grammar:

$S \rightarrow S+S$

$S \rightarrow S-S$

$S \rightarrow (S)$

$S \rightarrow a$

Input string:

$a1-(a2+a3)$

Parsing Algorithms

Stack contents	Input string	Actions
\$	a1-(a2+a3)\$	shift a1
\$a1	-(a2+a3)\$	reduce by $S \rightarrow a$
\$S	-(a2+a3)\$	shift -
\$S-	(a2+a3)\$	shift (
\$S-(	a2+a3)\$	shift a2
\$S-(a2	+a3)\$	reduce by $S \rightarrow a$
\$S-(S	+a3) \$	shift +
\$S-(S+	a3) \$	shift a3
\$S-(S+a3	) \$	reduce by $S \rightarrow a$
\$S-(S+S	) \$	shift)
\$S-(S+S)	\$	reduce by $S \rightarrow S+S$
\$S-(S)	\$	reduce by $S \rightarrow (S)$
\$S-S	\$	reduce by $S \rightarrow S-S$
\$S	\$	Accept

Parsing Algorithms

Advantages:

- ❖ Shift-reduce parsing is efficient and can handle a wide range of context-free grammars.
- ❖ It can parse a large variety of programming languages and is widely used in practice.
- ❖ It is capable of handling both left- and right-recursive grammars, which can be important in parsing certain programming languages.
- ❖ The parse table generated for shift-reduce parsing is typically small, which makes the parser efficient in terms of memory usage

Parsing Algorithms

Disadvantages:

- ❖ Shift-reduce parsing has a limited lookahead, which means that it may miss some syntax errors that require a larger lookahead.
- ❖ It may also generate false-positive shift-reduce conflicts, which can require additional manual intervention to resolve.
- ❖ Shift-reduce parsers may have difficulty in parsing ambiguous grammars, where there are multiple possible parse trees for a given input sequence.
- ❖ In some cases, the parse tree generated by shift-reduce parsing may be more complex than other parsing techniques.

Parsing Algorithms

Hypergraphs and Chart Parsing:

- ❖ A chart parser is a type of parser suitable for ambiguous grammars (including grammars of natural languages).
- ❖ It uses the dynamic programming approach—partial hypothesized results are stored in a structure called a chart and can be re-used.
- ❖ A hypergraph is a generalization of a graph in which an edge can join any number of vertices.
- ❖ **Example: CYK Algorithm**

Parsing Algorithms

CYK algorithm :

- ❖ The **CYK algorithm** is a parsing algorithm for context-free grammar. It is used to check if a particular string can be derived from the language generated by a given grammar.
 - ❖ It is also called the membership algorithm as it tells whether the given string is a member of the given grammar or not.
 - ❖ It was independently developed by three Russian scientists named **Cocke, Younger, and Kasami**, hence the name CYK!
 - ❖ In a CYK algorithm, the structure of grammar should be in Chomsky normal form.
 - ❖ In addition, the CYK algorithm uses the dynamic programming or table filling algorithm.
 - ❖ The grammar will be in CNF if each rule has one of the following forms:
 - $A \rightarrow BC$ (at most two variables on the right-hand side)
 - $A \rightarrow a$ (a single terminal on the right-hand side)
 - $S \rightarrow \emptyset$ (null string)
- If the given Grammar is not in the CNF form, convert it to the CNF form before applying the CYK Algorithm.

Parsing Algorithms

CYK algorithm :

Let w be the n length string to be parsed. And G represent the set of rules in our grammar with start state S .

1. Construct a table DP for size $n \times n$.
2. If $w = \epsilon$ (empty string) and $S \rightarrow \epsilon$ is a rule in G then we accept the string else we reject.
3. For $i = 1$ to n :
 For each variable A :
 We check if $A \rightarrow b$ is a rule and $b = w_i$ for some i :
 If so, we place A in cell (i, i) of our table.
4. For $l = 2$ to n :
 For $i = 1$ to $n-l+1$:
 $j = i+l-1$
 For $k = i$ to $j-1$:
 For each rule $A \rightarrow BC$: We check if (i, k) cell contains B and $(k + 1, j)$ cell contains C :
 If so, we put A in cell (i, j) of our table.
5. We check if S is in $(1, n)$:
 If so, we accept the string
 Else, we reject.

Parsing Algorithms

CYK algorithm :

To decide the membership of any given string, we construct a triangular table where-

- ❖ Each row of the table corresponds to one particular length of sub strings.
- ❖ Bottom most row corresponds to strings of length-1.
- ❖ Second row from bottom corresponds to strings of length-2.
- ❖ Third row from bottom corresponds to strings of length-3.
- ❖ Top most row from bottom corresponds to the given string of length-n.

Parsing Algorithms

CYK algorithm :

Let our grammar G be:

$S \rightarrow AB \mid BC$

$A \rightarrow BA \mid a$

$B \rightarrow CC \mid b$

$C \rightarrow AB \mid a$

We check if **baaba** is in $L(G)$:

Parsing Algorithms

CYK algorithm :

	b	a	a	b	a
b	{B}				
a		{A,C}			
a			{A,C}		
b				{B}	
a					{A,C}

	b	a	a	b	a
b	{B}	{S,A}	ϕ	ϕ	{S,A,C}
a		{A,C}	{B}	{B}	{S,A,C}
a			{A,C}	{S,C}	{B}
b				{B}	{S,A}
a					{A,C}

We observe that S is in the cell (1, 5), Hence, the string **baaba** belongs to **L(G)**.

Minimum Spanning Trees and Dependency Parsing:

- ❖ Dependency parsing can be approached as finding a Minimum Spanning Tree in a graph where the nodes represent words in a sentence, and the edges represent potential dependencies.
- ❖ Each word in the sentence is a node, and edges represent possible grammatical relations, such as subject-verb or modifier-head relationships.
- ❖ The goal is to find the MST of this graph, where the MST edges correspond to the most probable or relevant grammatical dependencies.
- ❖ Once the MST is identified, it can be used to construct the dependency parse tree for the sentence.

Minimum Spanning Tree-based : Start with a fully-connected directed graph and find a Minimum Spanning Tree. Then Subtract its score from all incoming edges. Contracts nodes if there are cycles and recursively computes MST.

Parsing Algorithms

Minimum Spanning Trees and Dependency Parsing:

- ❖ Connects all vertices with the minimum possible number of edges
- ❖ Has the maximum weight (summation of all edges scores involved) amongst all the possible spanning trees that connect all vertices.
- ❖ The weights of edges should be relative i.e if we perform any operation on the weights, the MST shouldn't be affected
- ❖ Each vertex should have only one incoming node in an MST
- ❖ Should have no cycle

Parsing Algorithms



Learning from Experience, Sharing with Passion

*For example: For '**book that flight**', we feed the model with*

book $\rightarrow$ that,

that $\rightarrow$ book,

that $\rightarrow$ flight,

flight $\rightarrow$ that,

book $\rightarrow$ flight &

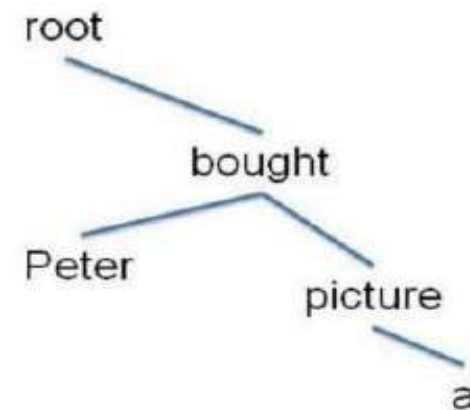
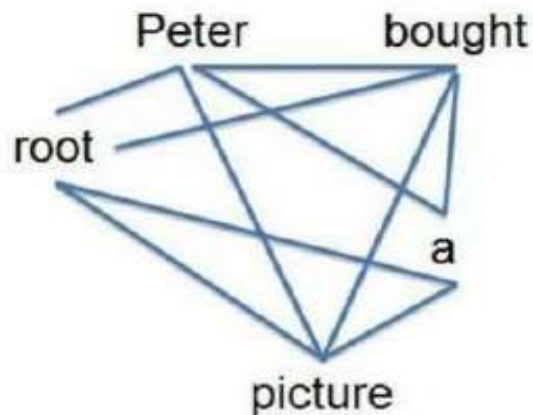
flight $\rightarrow$ book.

Their respective scores are predicted & assigned to respective edges for graph G.

<https://www.youtube.com/watch?v=JChpg5xUdL8>

Parsing Algorithms

This is equivalent to finding a spanning tree in the complete graph containing all possible arcs



Parsing Algorithms

- $$\frac{\text{\# correct dependencies}}{\text{\# of dependencies}}$$
- LAS – labeled attachment score
- UAS – unlabeled attachment score
- Which is bigger?

